OSSP Practical Experiments

Arun Tej(2520030472)

Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

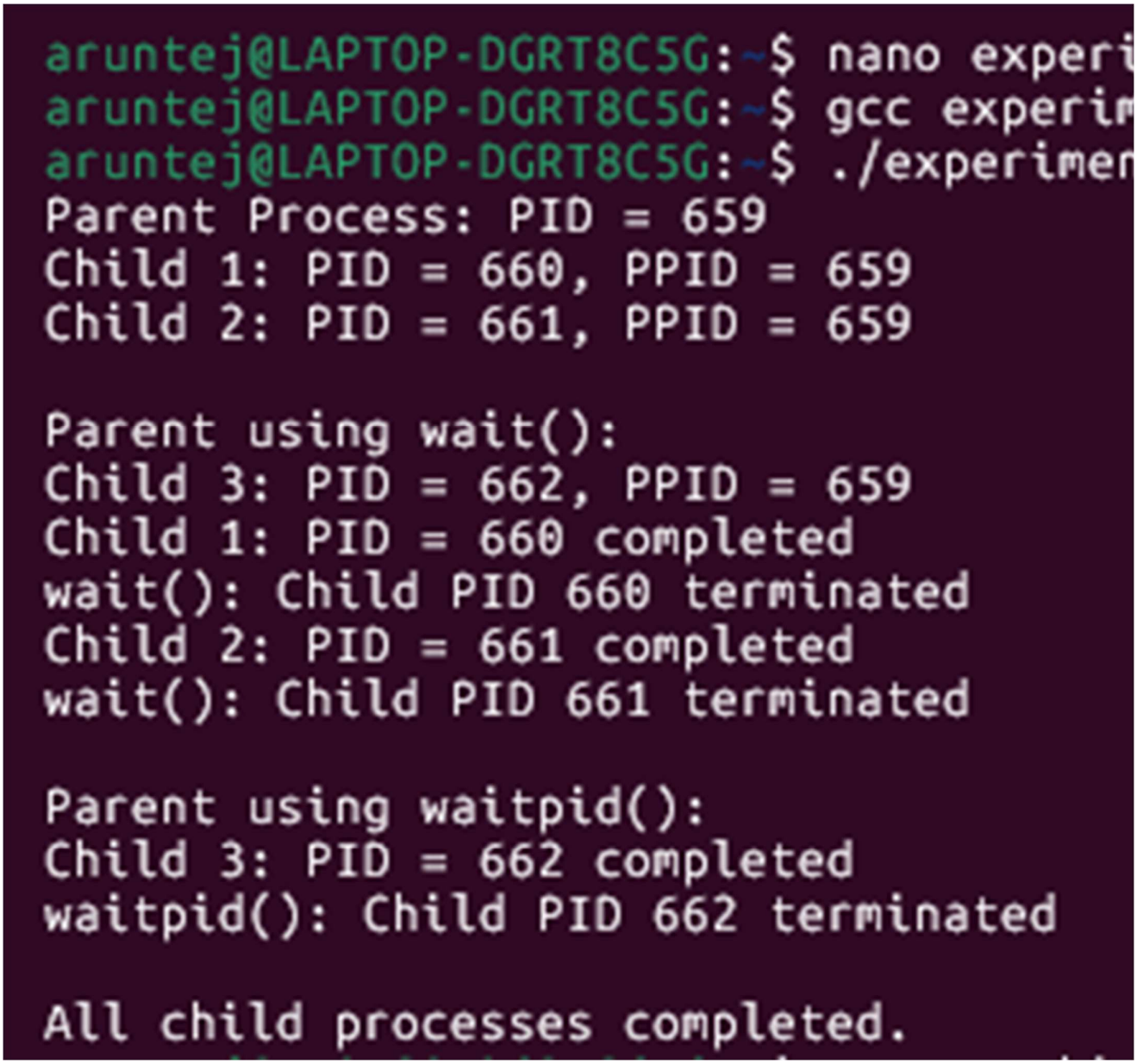
Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
```

```
int i;
pid_t pid, status;

printf("Parent Process: PID = %d\n", getpid());

// Create 3 child processes
for (i = 1; i <= 3; i++) {
   pid = fork();

   if (pid < 0) {
      perror("fork failed");
      exit(1);
   }

   if (pid == 0) {
      printf("Child %d: PID = %d, PPID = %d\n",
           i, getpid(), getppid());

      sleep(i * 2);
```

```
        printf("Child %d: PID = %d completed\n",
            i, getpid());

        exit(i);
    }
  }

  // Parent waits for any child
  printf("\nParent using wait():\n");

  for (i = 0; i < 2; i++) {
    pid = wait(&status);

    if (pid > 0) {
        printf("wait(): Child PID %d terminated\n",
pid);
    }
  }
```

```
    // Parent waits for a specific child
    printf("\nParent using waitpid():\n");

    while ((pid = waitpid(-1, &status, 0)) > 0) {
        printf("waitpid(): Child PID %d
terminated\n", pid);
    }

    printf("\nAll child processes completed.\n");

    return 0;
}
```



Zombie Process

#include <stdio.h>

#include <stdlib.h>

```
#include <unistd.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0) {
        printf("Child: PID = %d\n", getpid());
        printf("Child is terminating...\n");
        exit(0);
    }
    else {
        printf("Parent: PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
```

```
        printf("Parent is sleeping...\n");
        sleep(30);

        printf("Parent exiting...\n");
    }

    return 0;
}
```